POLITECNICO DI BARI

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE
MASTER DEGREE IN AUTOMATION ENGINEERING

Mobile robotics

Prof. Luca De Cicco

Title:

Autonomous Navigation System for Vision-Based Human Target Following in ROS 2 using ultralytics YOLO model

Student:
Domenico D'Introno

ACADEMIC YEAR 2024-2025

Contents

1 Robot **2**

2 Environment **3**

3 YOLO Computer Vision Model **4**

4 Interfaces **5**

5 Custom Nodes **6**
5.1 Perceptor 6
5.2 Localizer 8
5.3 Navigator 8
5.4 Initial position publisher 8

6 Launch File **9**

1 Robot

2 Environment

3 YOLO Computer Vision Model

4 Interfaces

5 Custom Nodes

This section describes the design and implementation of the three custom ROS 2 nodes: the **Perceptor**, the **Localizer**, and the **Navigator**. Together, they form a pipeline allowing the robot to detect a human in its camera feed, to estimate their 3D position relative to the robot, and to autonomously navigate towards that position using the `Nav2` framework.

5.1 Perceptor

The *Perceptor* node is responsible for detecting humans in the camera image stream. It leverages the YOLOv8 neural network model to perform real-time object detection.

Logic pipeline

1. **Subscribes** to the raw RGB image topic `/camera/image_raw`.
2. Applies YOLO to detect the human target.
3. Computes the centroid of the detected bounding box.
4. **Publishes** the centroid as a custom message `CentroidCoords` on `/centroid_coords`.

Code analysis

```python
import rclpy
from rclpy.node import Node

from sensor_msgs.msg import Image
from doom_interfaces.msg import CentroidCoords

from cv_bridge import CvBridge

from ultralytics import YOLO

class Detector(Node):

    def __init__(self):
        super().__init__('perceptor')

        self.model = YOLO("yolov8m.pt")

        self.bridge = CvBridge()

        self.subscriber_ = self.create_subscription(
            Image,
            '/camera/image_raw',
            self.detect,
            10
        )

        self.publisher_ = self.create_publisher(
            CentroidCoords,
            '/centroid_coords',
            10
        )

```

```
        self.get_logger().info(f"Detection node ready \n")

    def detect(self, msg):

        # Convert a sensor_msgs\Image message in a format that can be used by
            ↪ the YOLO model
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='
            ↪ passthrough')

        # Detect a person (class : 0) inside the camera image, use gpu
        results = self.model(cv_image, device="cuda", classes=[0], verbose=
            ↪ False)

        # Check that a person has actually been detected, otherwise publish
            ↪ the 'error centroid'
        if len(results[0].boxes) > 0:
            [x_min, y_min, x_max, y_max] = results[0].boxes.xyxy[0]

            # Compute centroid (top 25% to avoid targeting the void space
                ↪ between the legs)
            x_centroid = int((x_min + x_max) / 2)
            y_centroid = int(y_min + (y_max-y_min) * 0.25)

            # Publish centroid
            centroid = CentroidCoords()
            centroid.u = x_centroid
            centroid.v = y_centroid
            centroid.header = msg.header
        else:
            centroid = CentroidCoords()
            centroid.u = -1
            centroid.v = -1
            centroid.header = msg.header

        self.publisher_.publish(centroid)




def main(args=None):
    rclpy.init(args=args)
    node = Detector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
```

The constructor initializes the `perceptor` node: it loads the YOLOv8 model, initializes the CV bridge, which converts between ROS2 Image messages and OpenCV images, subscribes to camera

topic, and sets up a publisher for the detected centroid coordinates. The `detect` callback function runs person detection on the received camera image using YOLO, extracts the bounding box centroid, and publishes it as `CentroidCoords`.
Note that the `model` function takes as arguments `device="cuda"` to specify that the GPU will be used to run the model, plus, the `classes=[0]` argument narrows the model research to just persons.
An important design choice is to compute the person pixel centroid as the top 25% of its silhouette, fixing a bug where inconsistent depth measurements and mislocalization occurred when the centroid fell between the legs.
Also, if no person is detected, the node publishes an 'error centroid', meaning a centroid with both pixels coordinates set to `-1`. This strategy allows the downstream nodes to identify target loss.

5.2 Localizer

5.3 Navigator

5.4 Initial position publisher

6 Launch File